

HELPING HAND - CRAFTCULTURE PLATFORM

An Empowerment Platform for Handicraft Artisans with Profit Sharing, Job Opportunities & Material Donations

Course / Subject:	Flutter Application & Firebase Development
Project Category:	Full-Stack Mobile & Web Application
Database Backend:	Firebase Cloud Firestore & Local Reactive State
Frontend Framework:	Flutter SDK (Dart 3.x), Material Design 3
Academic Scope:	Complete Practical Syllabus Alignment (Practicals 1 - 24)

PROJECT DEVELOPMENT TEAM

Sr.	Student Name	Roll Number	Role / Contribution
1	Om Sawant	045	Lead Developer & State Architecture
2	Prity	150	UI/UX Designer & Frontend Screens

1. Executive Summary & Objectives

Handicraft artisans represent the rich cultural heritage of rural communities. However, traditional craftspeople face severe challenges such as exploitation by middlemen, lack of price transparency, insufficient raw material capital, and limited direct access to buyers. **Helping Hand - CraftCulture Platform** addresses these core issues through an integrated digital solution built with **Flutter** and **Firebase**.

Key Objectives of the Platform:

- **Artisan Empowerment:** Direct marketplace listing eliminating intermediary exploitation.
- **Transparent Profit Sharing:** Automated 70/15/15 profit distribution algorithm for direct artisan payouts, welfare funds, and raw material pools.
- **Job Opportunities Management:** Dedicated job board connecting craftspeople with bulk festival orders and restoration contracts.
- **Material & Financial Donations:** Crowd-supported raw material pool (clay, wood, yarn, tools) with live target progress tracking.
- **Dual-Role Navigation:** Instant role switching between Artisan Studio and Buyer Marketplace views.

2. System Architecture & Tech Stack

Frontend Architecture (Flutter SDK)

The frontend is developed using **Flutter 3.x** and **Dart**, providing a native cross-platform experience across Web, Android, iOS, and macOS. The user interface follows **Material Design 3** principles, incorporating custom typography, curated terracotta/gold themes, and micro-animations.

Key Frontend Patterns:

- **State Management:** Uses ChangeNotifier and InheritedNotifier (CraftStoreProvider) for reactive UI updates without third-party boilerplate.
- **Responsive Layouts:** Utilizes MediaQuery and GridView.builder to dynamically adjust grid columns across screen viewports.
- **Motion & Transitions:** Implements Hero image tags and AnimatedContainer for smooth visual feedback during route navigation.
- **Form Controls & Validation:** Employs GlobalKey<FormState> and custom validators for numeric input verification.

Backend Architecture (Firebase Suite)

The backend layer leverages **Firebase** services to provide scalable cloud data management and secure user authentication.

Firebase Service	Implementation & Usage in Project
------------------	-----------------------------------

Firestore Core	Initializes cross-platform configurations via <code>DefaultFirestoreOptions</code> .
Cloud Firestore	NoSQL Document Database managing products, jobs, and donations collections with real-time snapshots().
Firestore Auth	User registration and login using <code>createUserWithEmailAndPassword()</code> and <code>signInWithEmailAndPassword()</code> .
Offline Fallback	Robust try-catch wrapper in <code>FirestoreService</code> enabling seamless local mock mode execution if offline.

3. Core Functional Modules

Module 1: 70/15/15 Profit Sharing Algorithm

When an item is purchased, the platform automatically calculates the Net Profit (Selling Price minus Raw Material Cost) and splits it into three transparent streams:

- 1. **70% Direct Artisan Payout:** Transferred directly to the artisan's account.
- 2. **15% Welfare Fund:** Allocated to community healthcare and emergency support.
- 3. **15% Raw Material Pool:** Subsidizes raw materials (clay, wood, yarn) for underprivileged craftspeople.

Formula Implementation Code Snippet:

```
final netProfit = (price - materialCost).clamp(0.0, double.infinity); final artisanCut = netProfit * 0.70; final welfareCut = netProfit * 0.15; final materialCut = netProfit * 0.15;
```

Module 2: Artisan & Buyer Dashboards

- **Artisan Dashboard:** Displays cumulative sales revenue, direct 70% earnings, welfare pool contributions, and active listings management.
- **Buyer Dashboard:** Features product catalog search, category filtering, digital impact purchase receipt dialogs, and contribution history.

Module 3: Job Opportunity Board & Donations

- **Job Board:** Artisans view required craft skills, duration, and stipend budgets for bulk orders, submitting single-tap applications.
- **Donation System:** Enables crowd contributions of money or raw materials (clay, wood, yarn, machinery) with progress tracking.

4. Syllabus Mapping Matrix (Practicals 1 - 24)

Practical Topic	Description & Concept Covered	Implementation Location
P1 - P3	Setup, IDE Tools, main(), MaterialApp, Basic Widgets	main.dart
P4	User Inputs, Forms, GlobalKey Validation	add_product_screen.dart
P5 & P8	UI Layouts (GridView, Card, Stack, Expanded)	product_catalog_screen.dart
P6 & P9	Custom Themes, Palette & Fonts (ThemeData)	theme/app_theme.dart

P7	Stateful vs Stateless Widget Lifecycle	product_detail_screen.dart
P10	App Navigation, Navigator, BottomNavBar, Drawer	screens/home_screen.dart
P11	Animations & Motion (Hero, AnimatedContainer)	widgets/product_card.dart
P12	Interactive Widgets (ChoiceChip, Dropdown, Radio)	donate_screen.dart
P13	Dialogs, Alerts & SnackBars (AlertDialog)	job_opportunities_screen.dart
P14	State Management (ChangeNotifier, InheritedNotifier)	craft_store_provider.dart
P15 & P16	Data Modeling & JSON Parsing (fromJson/toJson)	models/product_model.dart
P17 & P18	Database (Cloud Firestore) & Auth (Firebase Auth)	services/firebase_service.dart
P22	Responsive & Adaptive UI (MediaQuery)	product_catalog_screen.dart

5. Viva Exam Preparation Guide & Q&A;

Below are key technical questions and answers designed for the oral Viva examination:

Q1: What is the main entry point and state architecture of your app?

Answer: The entry point is `void main()` in `main.dart`. State management is handled via `ChangeNotifier` and `InheritedNotifier` (`CraftStoreScope`), which trigger reactive rebuilds using `notifyListeners()`.

Q2: How does your profit sharing algorithm work?

Answer: The algorithm computes $\text{Net Profit} = \text{Price} - \text{RawMaterialCost}$, allocating 70% directly to the artisan, 15% to a community welfare fund, and 15% to a raw material pool.

Q3: How are Cloud Firestore and Firebase Auth integrated?

Answer: `FirebaseService` initializes `DefaultFirebaseOptions.currentPlatform`, using Firestore collections ('products', 'analytics') for NoSQL storage and `FirebaseAuth` for user sign-in/sign-up with fallback handling.

Q4: How does Form validation work in your project?

Answer: Forms use a `GlobalKey`. Calling `_formKey.currentState!.validate()` triggers validator functions on every child `TextFormField` before executing actions.

Q5: Why did you use `GridView.builder` instead of `GridView.count`?

Answer: `GridView.builder` lazily builds items as they scroll into view, drastically improving memory efficiency compared to instantiating all grid items upfront.

End of Official Academic Project Report